

Resumo de Python — Dicionários, def e estruturas do material

Material de revisão baseado nos arquivos enviados: aula de Dicionários, exercício do menu e arquivo Bhaskara.py.

1. Dicionários

Dicionários armazenam pares chave:valor (key:value). São mutáveis e são delimitados por chaves.

```
dicionario = {"nome": "Carlos", "idade": 21, "curso": "DS"}
```

Macete: pense sempre em CHAVE → VALOR. O dois-pontos separa a chave do valor.

2. Criar um dicionário

Dicionário vazio:

```
d = {}  
d = dict()
```

Com valores:

```
d = {"nome": "João", "idade": 20}
```

3. Acessar, modificar e adicionar

```
d["nome"]  
d.get("nome")  
d["idade"] = 21  
d["nota"] = 8.5
```

d["chave"] gera KeyError se a chave não existir. d.get("chave") retorna None quando a chave não existe.

A mesma atribuição serve para modificar uma chave existente ou criar uma nova.

4. Remover itens

```
d.pop("curso")  
del d["idade"]  
d.clear()  
del d
```

Macete: pop tira um item, clear limpa o conteúdo e del pode remover uma chave ou a própria variável.

5. Métodos importantes

Método	Para lembrar	Exemplo
.get()	pega um valor	d.get("nome")
.keys()	pega as chaves	d.keys()
.values()	pega os valores	d.values()
.items()	pega chave + valor	d.items()
.update()	atualiza	d.update({"idade": 22})
.pop()	remove uma chave	d.pop("curso")

<code>.clear()</code>	limpa tudo	<code>d.clear()</code>
-----------------------	------------	------------------------

6. Percorrendo dicionários

Quando você precisa da chave e do valor, use `.items()`:

```
for chave, valor in d.items():
    print(chave, valor)
```

Macete: se aparecer `for` chave, valor, pense automaticamente em `d.items()`.

7. Dicionários de dicionários

```
turma = {
    "aluno1": {"nome": "João", "nota": 7.5},
    "aluno2": {"nome": "Maria", "nota": 9.2}
}

print(turma["aluno2"]["nota"])
```

Leia os colchetes da esquerda para a direita: primeiro entra no `aluno2` e depois pega a `nota`.

8. Dicionários com listas

```
alunos = {
    "João": [7.5, 8.0, 9.0],
    "Maria": [9.2, 8.8, 9.5]
}
```

A chave identifica o aluno e a lista guarda as notas. Para calcular média: `sum(notas) / len(notas)`.

9. Lista de dicionários

```
tabela = [
    {"nome": "João", "idade": 20},
    {"nome": "Maria", "idade": 21}
]
```

Macete visual: `[ ]` representa lista; `{ }` representa dicionário; `[{ }]` representa uma lista de dicionários.

No material, registros são adicionados com `tabela.append(pessoa.copy())`, preservando uma cópia do registro naquele momento.

10. Resumo sobre def — funções e procedimentos

A palavra-chave `def` é usada para definir uma função. O material de Dicionários usa subalgoritmos para separar responsabilidades, como criar dados, calcular médias e exibir registros.

Estrutura básica:

```
def nome_da_funcao(parametro):  
    instrucoes
```

Quando a função precisa devolver um resultado, usa-se `return`:

```
def calcular_media(notas):  
    return sum(notas) / len(notas)
```

Macete: `def` = define uma função; `return` = devolve o resultado.

11. Parâmetros e tipos

O material apresenta anotações de tipo nos parâmetros e no retorno:

```
def calcular_media(notas: list) -> float:  
    return sum(notas) / len(notas)
```

`notas: list` indica o tipo esperado do parâmetro. `-> float` indica o tipo esperado do retorno.

Outro formato mostrado é `def exibir_aluno(nome: str, notas: list) -> None:`. O retorno `None` representa uma função usada para executar uma ação sem devolver um valor útil.

12. Funções usadas com dicionários

Uma vantagem de separar o programa em funções é dividir o problema em partes menores.

```
def exibir_aluno(nome: str, notas: list) -> None:  
    media = calcular_media(notas)  
    print(f"Aluno: {nome}")  
    print(f"Média: {media:.2f}")
```

Uma função pode chamar outra função. No material, `listar_alunos` percorre o dicionário e chama `exibir_aluno` para cada registro.

13. while, break e continue — Bhaskara

O arquivo `Bhaskara.py` usa `while True` para repetir o programa até encontrar `break`.

```
while True:  
    ...  
    if condicao:  
        continue  
    ...  
    break
```

`continue` pula para a próxima repetição do loop. `break` encerra o loop.

14. input, int e operações matemáticas

```
a = int(input("digite o valor de a: "))
```

`input` recebe texto; `int` converte para inteiro. No Bhaskara, Delta é calculado com $(b^2) - 4ac$. Em Python, `**` representa potência e `delta**0.5` representa a raiz quadrada.

15. f-string

```
print(f"Delta = {delta}, X1 = {x1}, X2 = {x2}")
```

Macete: coloque f antes das aspas e a variável entre { }. Exemplo: f"Nome: {nome}".

16. Menu do exercício

O exercício pede um menu para inicializar, modificar, remover e listar informações de um dicionário.

```
dicionario = {}

while True:
    print("0 - Sair")
    print("1 - Inicializar o dicionario")
    print("2 - Modificar um value")
    ...
    opcao = int(input("Escolha: "))

    if opcao == 0:
        break
```

A ideia é: criar o dicionário, mostrar o menu, receber a opção, executar a operação escolhida e voltar ao menu. A opção 0 encerra com break.

17. Cola de revisão

Objetivo	Sintaxe
Dicionário vazio	d = {}
Criar item	d["chave"] = valor
Acessar	d["chave"]
Acessar com segurança	d.get("chave")
Chaves	d.keys()
Valores	d.values()
Chave + valor	d.items()
Remover	d.pop("chave") / del d["chave"]
Limpar	d.clear()
Percorrer	for k, v in d.items():
Função	def nome(parametro):
Retornar	return resultado
Loop infinito	while True:
Próxima repetição	continue
Sair do loop	break
Potência	x ** 2
Raiz quadrada	x ** 0.5
Interpolação	f"texto {variavel}"

Prioridade para revisão: d["chave"], d.get(), d.keys(), d.values(), d.items(), d.pop(), d.clear(), for com .items(), dicionários aninhados, listas de dicionários, def, return, while True, break, continue, ** e f-strings.